

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

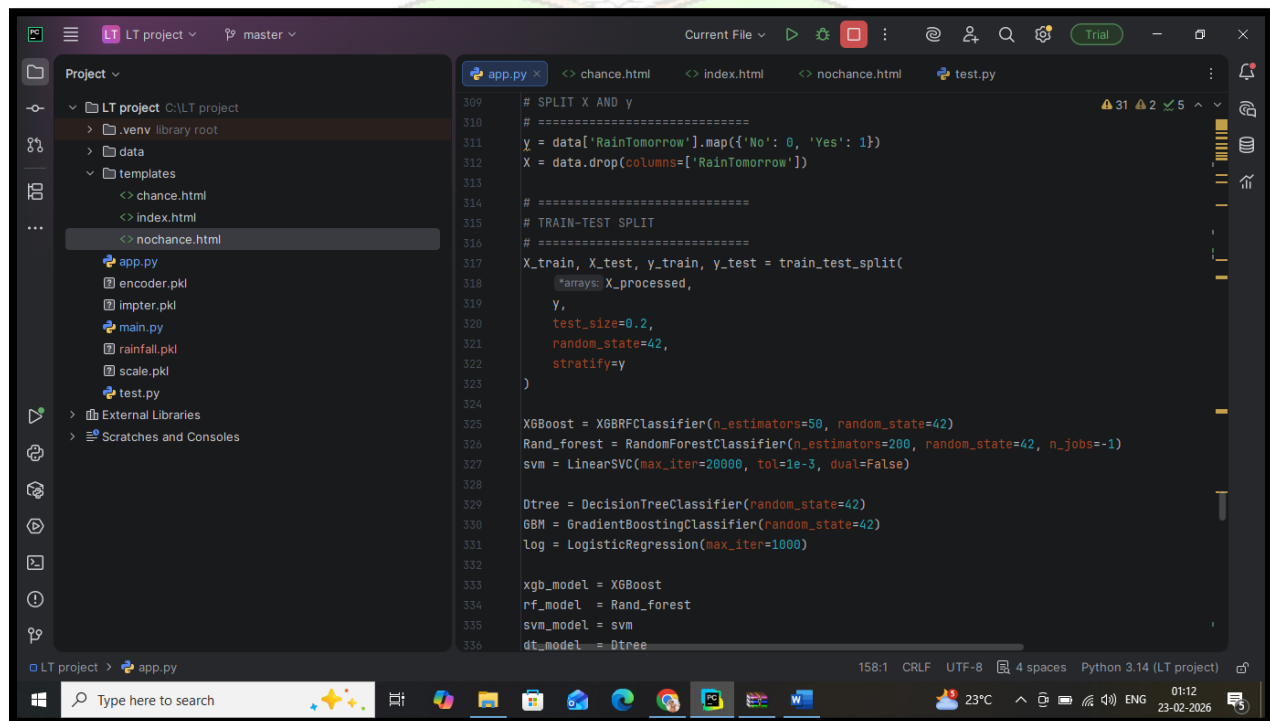
DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rainfall Data in India for Agriculture
MAXIMUM MARKS	4 MARKS

6.4 - MODEL BUILDING

Training and Testing the Model

Steps in Building the model:

- Initialize the model.
- Fit the models with x_train and y_train.
- Predict the y_train values and calculate the accuracy.
- Predict the y_test values and calculate the accuracy.



```
309 # SPLIT X AND y
310 # =====
311 y = data['RainTomorrow'].map({'No': 0, 'Yes': 1})
312 X = data.drop(columns=['RainTomorrow'])
313 # =====
314 # TRAIN-TEST SPLIT
315 # =====
316 X_train, X_test, y_train, y_test = train_test_split(
317     X, y,
318     test_size=0.2,
319     random_state=42,
320     stratify=y
321 )
322
323 XGBoost = XGBRFClassifier(n_estimators=50, random_state=42)
324
325 Rand_forest = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
326
327 svm = LinearSVC(max_iter=20000, tol=1e-3, dual=False)
328
329 Dtree = DecisionTreeClassifier(random_state=42)
330 GBM = GradientBoostingClassifier(random_state=42)
331 log = LogisticRegression(max_iter=1000)
332
333 xgb_model = XGBoost
334 rf_model = Rand_forest
335 svm_model = svm
336 dt_model = Dtree
```

- We have to paste code.

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

initializing all the models and predicting for better accuracy

```
[ ]: #Models initialization of the models
XGBoost = xgboost.XGBRFClassifier()
Rand_forest = sklearn.ensemble.RandomForestClassifier()
svm = sklearn.svm.SVC()
Dtree = sklearn.tree.DecisionTreeClassifier()
GBM = sklearn.ensemble.GradientBoostingClassifier()
log = sklearn.linear_model.LogisticRegression()

[ ]: # fitting the model
XGBoost.fit(x_train,y_train)
Rand_forest.fit(x_train,y_train)
svm.fit(x_train,y_train)
Dtree.fit(x_train,y_train)
GBM.fit(x_train,y_train)
log.fit(x_train,y_train)

[ ]: # predicting the train values
p1 = XGBoost.predict(x_train)
p2 = Rand_forest.predict(x_train)
p3 = svm.predict(x_train)
p4 = Dtree.predict(x_train)
p5 = GBM.predict(x_train)
p6 = log.predict(x_train)

[ ]: #checking the accuracy score
print("xgboost:",metrics.accuracy_score(y_train,p1))
print("Rand_forest:",metrics.accuracy_score(y_train,p2))
print("svm:",metrics.accuracy_score(y_train,p3))
print("Dtree:",metrics.accuracy_score(y_train,p4))
print("GBM:",metrics.accuracy_score(y_train,p5))
print("log:",metrics.accuracy_score(y_train,p6))
```

MODEL EVALUATION:

1. Accuracy score:

$$\text{Accuracy} = \frac{\text{Number of Correct predictions}}{\text{Total number of predictions made}}$$

- We have to paste the code for the value.

```
[40]: print("xgboost:",metrics.accuracy_score(y_test,t1))
      print("Rand_forest:",metrics.accuracy_score(y_test,t2))
      print("svm:",metrics.accuracy_score(y_test,t3))
      print("Dtree:",metrics.accuracy_score(y_test,t4))
      print("GBM:",metrics.accuracy_score(y_test,t5))
      print("log:",metrics.accuracy_score(y_test,t6))
```

```
xgboost: 0.8420478919793242
Rand_forest: 0.8569218326945391
svm: 0.8525967861035901
Dtree: 0.7837476704525476
GBM: 0.8499947255529379
log: 0.8418369140968388
```

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

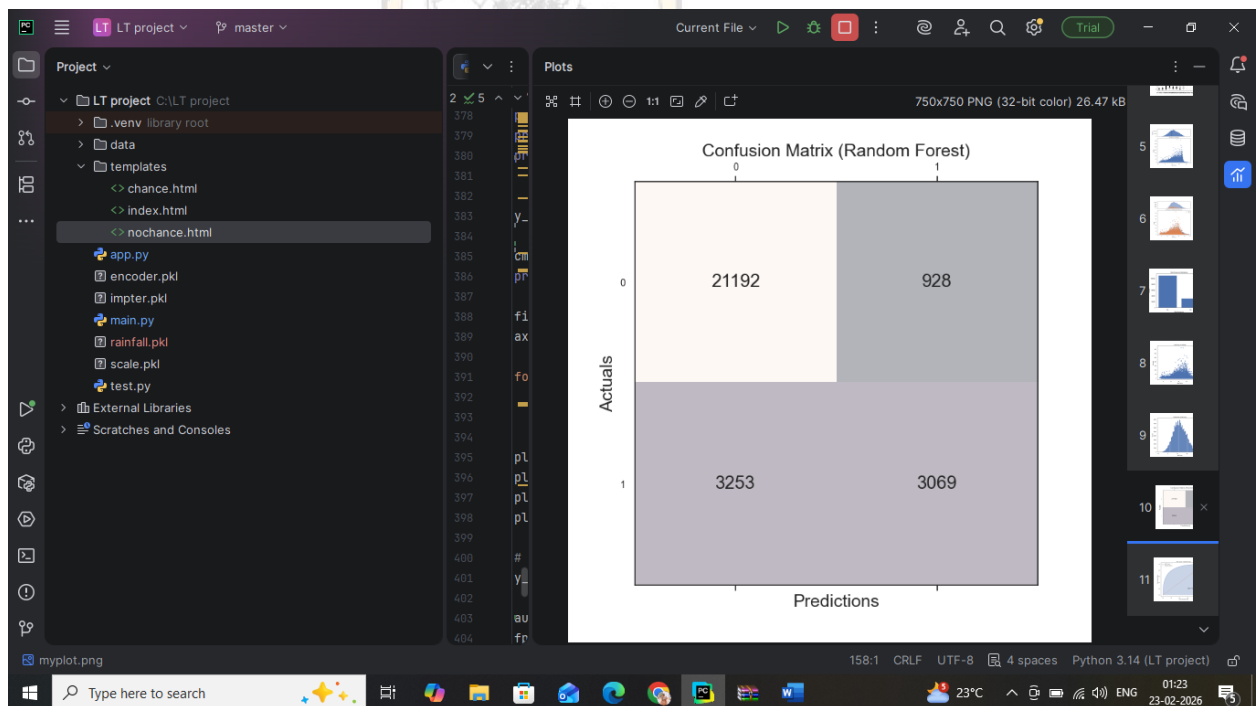
2. Confusion Matrix:

- It is a matrix representation of the results of any binary testing.

```
384 cm = confusion_matrix(y_test, y_pred)
385 print(cm)
386
387 fig, ax = plt.subplots(figsize=(7.5, 7.5))
388 ax.matshow(cm, alpha=0.3)
389
390 for i in range(cm.shape[0]):
391     for j in range(cm.shape[1]):
392         ax.text(j, i, cm[i, j], va='center', ha='center', size='xx-large')
393
394 plt.xlabel(xlabel: "Predictions", fontsize=18)
395 plt.ylabel(ylabel: "Actuals", fontsize=18)
396 plt.title(label: "Confusion Matrix (Random Forest)", fontsize=18)
397 plt.show()
398
399
```

- We have to paste the above code

Output:



Gayathri Educational Society
ESTD-2001

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

3. Roc Auc Curve:

- We have to paste the below code for getting the roc auc curve.

```
] : print(conf_matrix)
print("Accuracy:",Accuracy)
print("Precesion:",Precesion)
print("Recall:",Recall)
print("F1-score:",F1_score)

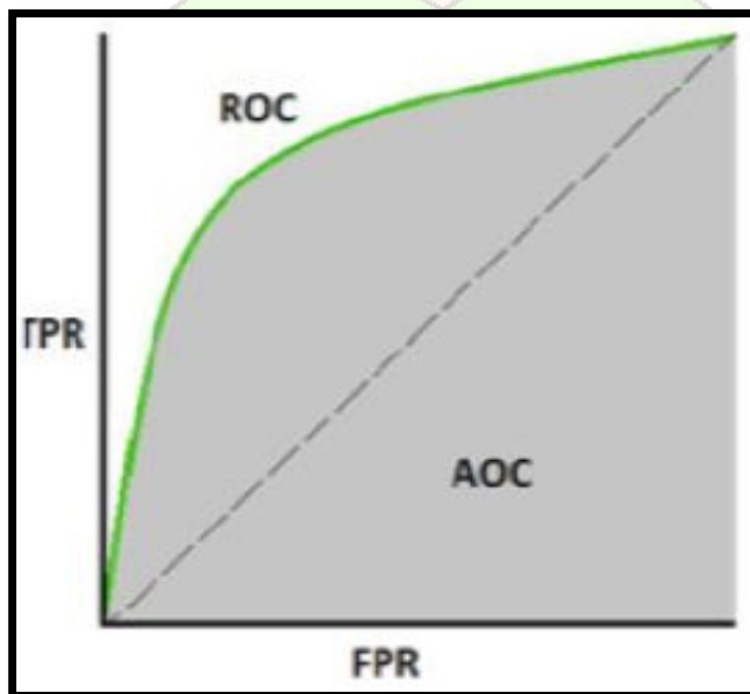
...

] : auc = metrics.roc_auc_score(y_test,y_pred)

fpr, tpr, thresholds = metrics.roc_curve(y_test,y_pred)

plt.figure(figsize=(12, 10), dpi=80)
plt.axis('scaled')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.title("AUC & ROC Curve")
plt.plot(fpr, tpr, 'v')
plt.fill_between(fpr, tpr, facecolor='blue', alpha=0.8)
plt.text(1, 0.05, 'AUC = %0.4f' % auc, ha='right', fontsize=10, weight='bold', color='black')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```

Output:



EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

Save the Model:

- After building the model we have to save the model.
- We have to paste the below code for pkl files.

```
# 1. Save the actual model you trained
# In your script, this was named 'Rand_forest'
pickle.dump(Rand_forest, open('rainfall.pkl', 'wb'))

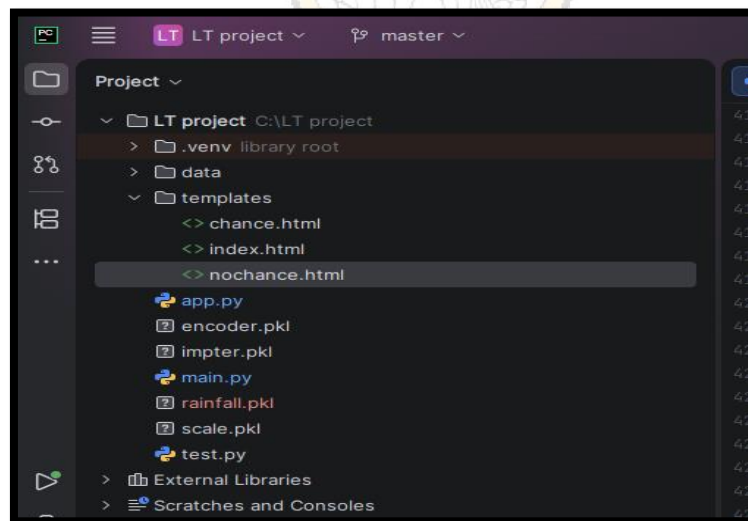
# 2. Save the preprocessor
# This replaces 'le', 'imp_mode', and 'sc' because your ColumnTransformer
# already contains the Imputer, Scaler, and OneHotEncoder combined.
pickle.dump(preprocessor, open('encoder.pkl', 'wb'))

# 3. Save the categorical imputer (Optional)
# If you specifically need the categorical imputer you named 'imp_mode'
pickle.dump(imp_mode, open('impter.pkl', 'wb'))

# This object contains your scaling logic
pickle.dump(preprocessor, open('scale.pkl', 'wb'))
```

Output:

- After running the code the pkl files created like below.



Gayathri Educational Society
ESTD-2001